ÜBERBLICK

Einstieg für Neueinsteiger

Was das Projekt enthält, in welcher Reihenfolge man liest und wo man bei einer Frage weiterliest

Dieses PDF gibt die Quelle im Wortlaut wieder; Listen, Tabellen und Codeblöcke entsprechen der Quelle, weich umbrochene Zeilen sind zu Absätzen zusammengeführt.

Quelle	C8205-Einstieg_fuer_Neueinsteiger.md	0.01-002
Stand	13.10.2026	
Umfang	10 Seiten, A4 hoch	
Erzeugt mit	erzeuge_einstieg_pdf.py	

> Maßgeblich bleibt die Quelldatei. Dieses PDF wird nicht automatisch nachgeführt: Es ist aktuell, solange die Version oben mit der Versionszeile der Quelldatei übereinstimmt.

C 8205 – Einstieg für Neueinsteiger

Quelle: C8205-Einstieg_fuer_Neueinsteiger.md · im Wortlaut der Quelle

Inhaltsverzeichnis

1. Was ist dieses Projekt? . . . 3
1.1 Das Regelwerk ist ausführbar . . . 3
1.2 Zwei Wege: Python-Interpreter und Claude-Interpreter . . . 4

2. Die drei Teilbereiche . . . 4
2.1 Interpreter . . . 4
2.2 Assembler . . . 4
2.3 Bausteininterpretationssystem BIS . . . 4

3. Die wichtigste Unterscheidung . . . 5
Ebene A – der historische Rechner . . . 5
Ebene B – die Rekonstruktion . . . 5
Ebene C – die heutige Simulation . . . 5

4. Wo sollte ich anfangen? . . . 5

5. Was muss ich nicht sofort lesen? . . . 6

6. Die Dokumentation als Landkarte . . . 6

7. Der wichtigste technische Zusammenhang . . . 7

8. Wie lese ich einen einzelnen Befehl? . . . 7

9. Wie lese ich einen Test? . . . 7

10. Rekonstruktion und historische Sicherheit . . . 8

11. Wenn du selbst am Projekt arbeiten möchtest . . . 8
„Ich möchte nur einen Test ausführen.“ . . . 8
„Ich möchte einen Maschinenbefehl verstehen.“ . . . 8
„Ich möchte den Python-Interpreter ändern.“ . . . 8
„Ich möchte einen neuen Test schreiben.“ . . . 9
„Ich möchte Assemblerprogramme schreiben.“ . . . 9
„Ich möchte das BIS verstehen.“ . . . 9
„Ich möchte eine .md-Datei ändern.“ . . . 9

12. Was sollte ich beim ersten Lesen verstehen? . . . 9

13. Wenn du irgendwo nicht weiterkommst . . . 9

14. Kurzfassung . . . 10

C 8205 – Einstieg für Neueinsteiger

Version: 0.01-002

Zweck dieses Dokuments (Orientierung für neue Leser): Dieses Dokument richtet sich an Leser, die das C-8205-Projekt zum ersten Mal sehen. Es erklärt nicht alle technischen Einzelheiten des Rechners. Stattdessen zeigt es, **was im Projekt enthalten ist, in welcher Reihenfolge man die Dokumentation lesen sollte und wo man bei einer konkreten Frage weiterliest.**

Wenn du das Projekt bereits kennst, kannst du dieses Dokument überspringen.

1. Was ist dieses Projekt?

Das Projekt rekonstruiert und simuliert den historischen Kleinrechner **Cellatron C 8205** (1968 in der DDR gebaut).

Wenn du das Projekt über GitHub gefunden hast, kennst du vermutlich schon die `README.md` (deutsch) bzw. `README.en.md` (englisch). Sie fasst das Projekt auf einer Seite zusammen und verweist auf die Dokumente. Dieses Dokument führt von dort aus weiter.

Der wichtigste Teil des Projekts ist nicht eine vereinfachte Nachprogrammierung der Maschine. Stattdessen wird versucht, die historische Befehlsverarbeitung möglichst genau nachzubilden.

Dazu wurde aus den verfügbaren Unterlagen ein **Regelwerk** rekonstruiert.

Dieses Regelwerk beschreibt unter anderem:

- welche Knoten es im Befehlsablauf gibt,
- welche Eingänge und Ausgänge die Knoten besitzen,
- welche Anweisungen an den Knoten ausgeführt werden,
- wie Register und Recheneinheiten wirken,
- wie Befehle verarbeitet werden,
- wie die Peripheriegeräte eingebunden sind.

Der Python-Interpreter führt dieses Regelwerk aus.

Vereinfacht sieht der Zusammenhang so aus:

```
historische Unterlagen
        │
        ▼
rekonstruiertes Regelwerk
(.md-Dateien in markdown-dateien/)
        │  Knotengraph und Variablentabelle T2
        │  werden bei jedem Start gelesen
        ▼
Python-Interpreter
        │
        ▼
Testlauf → Testprotokoll
```

1.1 Das Regelwerk ist ausführbar

Das Regelwerk ist keine Beschreibung, die neben dem Programm steht. Der Interpreter liest es bei jedem Start selbst ein. Zwei Dateien in `markdown-dateien/` sind deshalb nicht nur Dokumentation, sondern Eingabe des Interpreters:

Datei	Inhalt	Wird gelesen als
`C8205-Interpreter-Eingänge_der_Knoten.md`	Knoten, Typen, Ein- und Ausgänge, Verzweigungen	Knotengraph
`C8205-Interpreter-Beschreibung_der_Knoten.md`	Bedeutung jeder ANWEISUNG-nnn	Variablentabelle T2

Den Ordner bestimmt der Schalter `--mdverz` (Vorgabe: `markdown-dateien/` neben `c8205_interp.py`). Nur die **Wirkung** jeder einzelnen ANWEISUNG ist von Hand als Python-Funktion hinterlegt. Fehlt zu einer ANWEISUNG eine Funktion, meldet der Interpreter das als **Stub**, statt zu raten.

Daraus folgt:

- Eine Änderung an diesen beiden Dateien ändert das Verhalten der Simulation, ohne dass Python-Code angefasst wird.

- Bei einem unerwarteten Testergebnis ist die Ursache deshalb nicht automatisch im Python-Code zu suchen (siehe Abschnitt 9).

1.2 Zwei Wege: Python-Interpreter und Claude-Interpreter

Dasselbe Testformat lässt sich auf zwei Wegen ausführen. Der **Python-Interpreter** (`c8205_interp.py`) arbeitet den Knotengraphen automatisch und reproduzierbar ab. Beim **Claude-Interpreter** erhält eine KI das komplette Regelwerk und dieselbe Testdatei und arbeitet den Graphen Knoten für Knoten von Hand ab. Er dient als unabhängige Gegenprobe.

Dieses Dokument behandelt den Python-Interpreter. Mehr zum Claude-Interpreter steht in `C8205-Interpreter-Projektueberblick.md`, Abschnitt 2, und in `C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 1b. Die Datei `baz-wirkungstabelle.md` ist ein Zwischenspeicher für den Claude-Interpreter und für den Einstieg nicht nötig.

2. Die drei Teilbereiche

2.1 Interpreter

Der Interpreter befindet sich hauptsächlich im Wurzelverzeichnis.

Er führt C-8205-Maschinencode anhand des rekonstruierten Knotengraphen aus.

> **Wichtig:** Der Python-Interpreter ist **nicht** die historische Software des C 8205. Er ist das heutige Simulationswerkzeug für die rekonstruierte Hardware.

Der wichtigste Einstieg ist:

`c8205_interp.py`

Die fachliche Beschreibung steht in:

`markdown-dateien/C8205-Interpreter-Python_Interpreter.md`

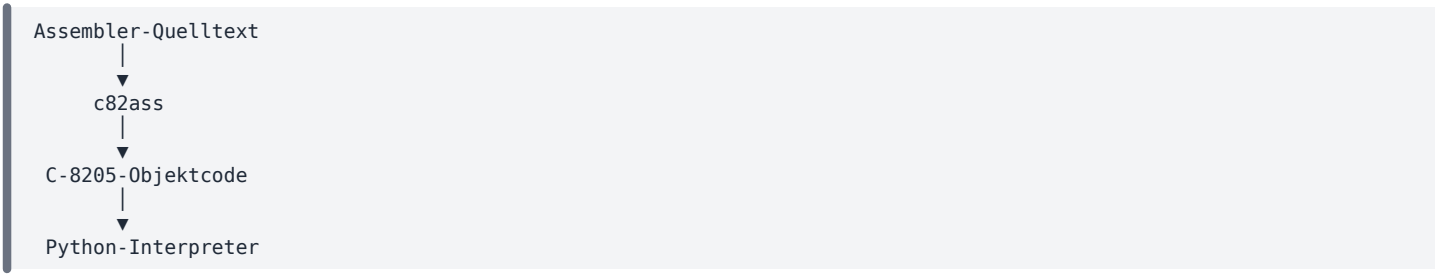
2.2 Assembler

Der Assembler (in C geschrieben) befindet sich unter:

`Assembler/`

`c82ass` übersetzt lesbaren Assembler-Quelltext in Objektcode für den C 8205.

Der grundlegende Ablauf ist:



Der Assembler ist ein eigenständiges Teilprojekt.

Wenn du dich zunächst nur für den Simulator interessierst, musst du die Assembler-Dokumentation **nicht als erstes lesen**.

Der Einstieg für den Assembler ist:

`Assembler/Dokumentation/C8205_Assembler_Dokumentation.md`

2.3 Bausteininterpretationssystem BIS

Unter

`Bausteininterpretiersystem BIS/`

befindet sich die Dokumentation des historischen Bausteininterpretationssystems.

Das BIS ist **kein Bestandteil des Python-Interpreters**.

Es handelt sich um Programmsysteme, die auf dem historischen C 8205 laufen bzw. dafür vorgesehen sind.

Wenn du zunächst verstehen möchtest, **wie der simulierte Rechner selbst funktioniert**, kannst du das BIS zunächst überspringen.

3. Die wichtigste Unterscheidung

Beim Lesen der Dokumentation begegnen dir drei verschiedene Ebenen.

Ebene A – der historische Rechner

Hier geht es um den tatsächlichen C 8205:

- Rechenwerk
- Leitwerk
- Register
- Magnettrommel
- Bedienpult
- Lochstreifen
- Schreibwerk
- Befehle

Ebene B – die Rekonstruktion

Hier wird beschrieben, wie aus den historischen Unterlagen ein ausführbares Regelwerk entstanden ist.

Das ist der **Knotengraph** mit der Variablentabelle T2.

Ebene C – die heutige Simulation

Hier geht es um Python, Dateien, Testskripte und virtuelle Geräte.

Diese drei Ebenen sollten beim Lesen möglichst nicht miteinander verwechselt werden.

4. Wo sollte ich anfangen?

Wenn du das Projekt zum ersten Mal öffnest, lies die Dokumentation **nicht einfach alphabetisch**.

Empfohlene Reihenfolge (alle Dateien außer diesem Dokument liegen in `markdown-dateien/`):

Schritt	Lies	Danach weißt du
1	dieses Dokument	was die verschiedenen Teile des Projekts überhaupt bedeuten
2	`C8205-Interpreter-Projektueberblick.md`	die Gesamtidee; das Dokument verweist auf die einzelnen Fachdokumente
3	`C8205-Interpreter-Überblick_und_Gesamtaufbau.md`	was der C 8205 technisch ist (du musst noch nicht jedes Detail verstehen)
4	`C8205-Projektstruktur.md`	welche Datei wo liegt und wofür sie zuständig ist (eine Landkarte, kein Lehrbuch)
5	`C8205-Interpreter-Datenstruktur_und_Verarbeitungseinheiten.md`	die wichtigsten Register und Datenstrukturen
6	`C8205-Interpreter-Befehlsliste.md`	welche Befehle der Rechner kennt
7	`C8205-Interpreter-Eingänge_der_Knoten.md` und `C8205-Interpreter-Beschreibung_der_Knoten.md`	der Kern der Rekonstruktion: wie aus einem Maschinenbefehl der tatsächliche Ablauf wird
8	`C8205-Interpreter-Python_Interpreter.md`	wie das heutige Python-Programm dieses Regelwerk ausführt
9	`C8205-Interpreter-Durchführung_eines_Tests.md`	wie du einen Testlauf nachvollziehst

5. Was muss ich nicht sofort lesen?

Beim ersten Durchgang kannst du diese Dokumente zunächst überspringen:

- `C8205-Interpreter-PDF.md`
- `C8205-Versionierung.md`
- `C8205-Interpreter-Lochstreifen-Hilfsprogramme.md`
- `Bausteininterpretiersystem BIS/markdown-dateien/C8205-Interpreter-Objektdatei_Listing.md`
- `C8205-Interpreter-Kodetabellen.md`
- `C8205-Interpreter-Magnettrommel.md`
- `C8205-Interpreter-Lochstreifenleser.md`
- `baz-wirkungstabelle.md`
- Assembler-Dokumentation
- BIS-Dokumentation

Diese Dokumente sind wichtig, aber **nicht alle sind Einstiegsliteratur**.

Sie werden später als Nachschlagewerk benötigt.

6. Die Dokumentation als Landkarte

Wenn du eine bestimmte Frage hast, gehe direkt zu dem passenden Dokument. Alle Dateien liegen in `markdown-dateien/`, sofern kein anderer Pfad angegeben ist. Die vollständige Dokumentenlandkarte steht in `C8205-Interpreter-Projektueberblick.md`, Abschnitt 6.

Frage	Hier weiterlesen
Was ist das Gesamtprojekt?	`C8205-Interpreter-Projektueberblick.md`
Wie ist das Projekt aufgebaut?	`C8205-Projektstruktur.md`
Was liegt in `Tests/`, `Test-Protokolle/` und `virtuelle_Geraete/`?	`C8205-Projektstruktur.md`
Wie funktioniert der Rechner grundsätzlich?	`C8205-Interpreter-Überblick_und_Gesamtaufbau.md`
Welche Register gibt es?	`C8205-Interpreter-Datenstruktur_und_Verarbeitungseinheiten.md`
Welche Befehle gibt es?	`C8205-Interpreter-Befehlsliste.md`
Wie sind die Knoten aufgebaut?	`C8205-Interpreter-Beschreibung_der_Knoten.md`
Welche Eingänge haben die Knoten?	`C8205-Interpreter-Eingänge_der_Knoten.md`
Wie funktioniert der Python-Interpreter?	`C8205-Interpreter-Python_Interpreter.md`
Was ist der Claude-Interpreter?	`C8205-Interpreter-Projektueberblick.md`, Abschnitt 2
Wie führe ich einen Test aus?	`C8205-Interpreter-Durchführung_eines_Tests.md`
Wie funktioniert die Magnettrommel?	`C8205-Interpreter-Magnettrommel.md`
Wie funktioniert der Lochstreifenleser?	`C8205-Interpreter-Lochstreifenleser.md`
Welche Zeichenkodierungen gibt es?	`C8205-Interpreter-Kodetabellen.md`
Wie sieht Objektcode aus?	`Bausteininterpretiersystem BIS/markdown-dateien/C8205-Interpreter-Objektdatei_Listing.md`
Wie funktioniert das Bedienpult?	`C8205-Interpreter-Bedienpult_mit_Schaltern.md`
Wie funktioniert der Assembler?	`Assembler/Dokumentation/C8205_Assembler_Dokumentation.md`
Was ist das BIS?	`Bausteininterpretiersystem BIS/markdown-dateien/C8205-Interpreter-BIS.md`

7. Der wichtigste technische Zusammenhang

Beim Lesen sollte man sich immer diesen Ablauf vor Augen halten:

```
C-8205-Programm
      │
      ▼
   Objektcode
      │
      ▼
 Python-Interpreter
      │
      ▼
 Befehlsverarbeitung
      │
      ▼
   Knotengraph
      │
      ├── Knoten
      ├── Eingänge
      ├── Anweisungen
      └── Übergänge
      │
      ▼
 Register / Speicher / Geräte
      │
      ▼
    Ergebnis
```

Der Knotengraph ist dabei der zentrale Punkt.

Der Interpreter soll nicht einfach nur eine abstrakte Operation wie

```
ADD → AC = AC + Speicher
```

ausführen, sondern die rekonstruierte Befehlsverarbeitung entsprechend dem Regelwerk nachvollziehen.

Der Interpreter bekommt dabei nie nur ein Programm, sondern einen **Testauftrag**: den Trommelinhalt (oktal), eine Tastenfolge als Ausgangszustand (z. B. `H G R`), Testmarken für Diagnoseausgaben und optional Prüfungen mit Sollwerten. Dadurch lassen sich Läufe als Regressionstests wiederholen. Alle Zeiten der echten Hardware (Wortzeit, Wartezeit der Trommel) sind in der Simulation 0; die Reihenfolge der Ereignisse bleibt erhalten.

8. Wie lese ich einen einzelnen Befehl?

Wenn du verstehen möchtest, wie ein bestimmter Maschinenbefehl funktioniert, solltest du nicht im Python-Quelltext anfangen. Das ist der Nachschlageweg für einen bestimmten Befehl; den Lernweg für das ganze Projekt beschreibt Abschnitt 4.

Empfohlener Weg:

```
1. Befehlsliste
       ↓
2. Beschreibung des Befehls
       ↓
3. relevante Knoten
       ↓
4. Knoteneingänge
       ↓
5. verwendete Register/Datenstrukturen
       ↓
6. Python-Implementierung der Anweisungen
       ↓
7. Test
```

So bleibt erkennbar, **was historische Rekonstruktion und was technische Implementierung ist**.

9. Wie lese ich einen Test?

Ein Test besteht nicht nur aus „Eingabe → Ausgabe“.

Der Test definiert einen Ausgangszustand und beschreibt, was während der Befehlsverarbeitung passieren soll.

Bei einem Fehler sollte deshalb zunächst gefragt werden:

1. War der Ausgangszustand richtig?

2. War der Trommelzustand richtig?
3. Waren Registerwerte richtig?
4. Wurde der richtige Befehl geladen?
5. Welcher Knoten wurde zuletzt ausgeführt?
6. Welche Anweisung war dort problematisch?
7. War die Regelwerksbeschreibung eindeutig?
8. Erst danach: Liegt ein Fehler im Python-Interpreter vor?

Diese Reihenfolge verhindert, dass ein Problem vorschnell dem Python-Code zugeschrieben wird.

Wo die Teile eines Tests liegen:

Ordner	Inhalt
`Tests/`	Testaufträge (meist `.txt`; der Assembler erzeugt `.test`)
`Test-Ergebnisvergleiche/`	Soll-Dateien (Namen auf `-ausgabe.txt` oder `_erwartet.txt`), mit denen Tests ihre Ausgabe vergleichen
`Test-Protokolle/`	Protokolle (`.log`) der Interpreterläufe
`virtuelle_Geraete/`	die virtuellen Geräte, je ein Unterordner: `Leser1/`, `Leser2/`, `Stanzer1/`, `SW-Ausgabe/`, `T-SW-Eingabe/`, `Trommel/`

10. Rekonstruktion und historische Sicherheit

Eine wichtige Besonderheit dieses Projekts ist die historische Rekonstruktion.

Beim Lesen sollte deshalb möglichst unterschieden werden zwischen:

```
historisch belegt
       ≠
rekonstruiert
       ≠
angenommen
       ≠
technische Implementierung
```

Wenn eine Stelle in den historischen Unterlagen nicht eindeutig ist, sollte die Implementierung diese Unsicherheit nicht unbemerkt in einen scheinbar sicheren Fakt verwandeln.

Das Projekt hat dafür einen festen Mechanismus: Unklare Stellen meldet der Interpreter als offenen Punkt („Stub“), statt sie stillschweigend zu „reparieren“.

Für Änderungen und Erweiterungen des Projekts ist diese Unterscheidung besonders wichtig.

11. Wenn du selbst am Projekt arbeiten möchtest

Je nachdem, was du tun möchtest, gibt es unterschiedliche Einstiegspunkte.

„Ich möchte nur einen Test ausführen.“

Lies:

`markdown-dateien/C8205-Interpreter-Durchführung_eines_Tests.md`

Danach brauchst du zunächst keine Assembler- oder BIS-Kenntnisse.

„Ich möchte einen Maschinenbefehl verstehen.“

Gehe vor wie in Abschnitt 8.

„Ich möchte den Python-Interpreter ändern.“

Lies zuerst:

`markdown-dateien/C8205-Interpreter-Python_Interpreter.md`

und danach die Dokumentation des betreffenden Regelwerks.

Beachte: Das Verhalten des Knotengraphen steht in den beiden Knoten-Dateien (Abschnitt 1.1). In Python liegt nur die Wirkung der einzelnen Anweisungen.

„Ich möchte einen neuen Test schreiben.“

Beginne mit:

`markdown-dateien/C8205-Interpreter-Durchführung_eines_Tests.md`

und schaue dir anschließend vorhandene Dateien unter:

`Tests/`

an.

„Ich möchte Assemblerprogramme schreiben.“

Beginne in:

`Assembler/Dokumentation/`

und arbeite anschließend mit den vorhandenen `.asm`-Beispielen.

„Ich möchte das BIS verstehen.“

Beginne unter:

`Bausteininterpretiersystem BIS/`

und nicht bei der Python-Interpreter-Dokumentation.

„Ich möchte eine .md-Datei ändern.“

Lies zuerst:

`markdown-dateien/C8205-Versionierung.md`

Jede Änderung an einer Datei erhöht ihren Build (`zzz` in `y.xx-zzz`) um 1. Haupt- und Unterversion erhöht nur der Projektinhaber auf ausdrückliche Aufforderung.

12. Was sollte ich beim ersten Lesen verstehen?

Du musst nach dem ersten Durchgang **nicht**:

- alle Knoten kennen,
- alle Befehle auswendig kennen,
- die komplette Hardware verstehen,
- den Assembler verstehen,
- das BIS verstehen.

Das erste Ziel ist viel kleiner:

> **Verstehen, wie die Teile zusammengehören.**

Wenn du diese fünf Aussagen verstanden hast, hast du den wichtigsten ersten Schritt geschafft:

1. Der C 8205 ist der historische Rechner.
2. Das Regelwerk (die .md-Dateien) beschreibt die rekonstruierte Befehlsverarbeitung.
3. Der Knotengraph ist der zentrale Ablauf dieser Befehlsverarbeitung und steht als Datei im Regelwerk.
4. Der Python-Interpreter liest diesen Knotengraphen und führt ihn aus.
5. Tests zeigen, ob die Rekonstruktion und ihre Implementierung wie erwartet funktionieren.

Alles Weitere kann anschließend schrittweise gelesen werden. **Erst danach** sollte man sich gezielt in Magnettrommel, Lochstreifen, Assembler, BIS und die einzelnen Sonderfälle vertiefen.

13. Wenn du irgendwo nicht weiterkommst

Die Dokumentation ist bewusst aufgeteilt.

Du solltest nicht versuchen, eine einzelne Datei vollständig zu verstehen.

Wenn beispielsweise in einer Beschreibung steht:

> „siehe Datenstruktur und Verarbeitungseinheiten“

ist das kein fehlender Inhalt, sondern ein Verweis auf die nächste Ebene.

Die Dokumentation funktioniert daher eher wie ein **technisches Nachschlagewerk mit Querverweisen** als wie ein einzelnes Lehrbuch.

14. Kurzfassung

Wenn du nur eine Sache aus diesem Dokument mitnimmst:

```
HISTORISCHER C 8205 → REKONSTRUKTION → REGELWERK → KNOTENGRAPH → PYTHON-INTERPRETER → TESTS
```

Lies von oben nach unten und gehe erst dann in die Details.

Für die meisten neuen Leser ist die beste Reihenfolge:

Projektüberblick → Gesamtaufbau → Projektstruktur → Datenstrukturen → Befehle → Knoten → Interpreter → Tests.

Assembler und BIS sind eigene Wege und müssen nicht verstanden werden, um den Interpreter zu verstehen.